Gold Price Prediction Using Commodity and Macroeconomic Indicators

Iteration 4 Progress Report: Machine Learning Model Development and Evaluation

EECE 5644: Introduction to Machine Learning and Pattern Recognition

Team: Gaurav Harish, Luke Sanders, Omar Gomaa

GitHub: https://github.com/khgaurav/macro-commodity-ml

1. Overview

This iteration implements, tunes, and evaluates three machine learning models: Lasso, XGBoost, and LSTM; to predict `tomorrow_gold_log_return`, the next-trading-day log return of gold, from 21 commodity, market, and macroeconomic features. The 5,016-row daily dataset (Jan 22, 2004 – Jul 13, 2026) and its preprocessing were built and documented in Iteration 3; this report covers only the modeling work completed in Iteration 4.

2. Algorithms Selected and Justification

Model	Why Selected
Lasso	Linear baseline with L1-driven feature selection; sparse, interpretable, and inexpensive to tune
XGBoost	Tree-based ensemble that captures nonlinear feature interactions without scaling, and provides native gain-based feature importance.
LSTM	The only model that consumes ordered 30-day sequences rather than single-day snapshots, allowing it to learn temporal patterns the other two cannot see.

3. Training and Testing Process

Splits are chronological to prevent any future information from leaking into training: 70% train / 15% validation / 15% test, identical across all three models.

Split	Rows	Date Range	Role
Training	3,511	2004-01-22 – 2020-07-13	Model fitting
Validation	752	2020-07-14 – 2023-07-10	Tuning / early stopping
Test	753	2023-07-11 – 2026-07-13	Final evaluation

4. Hyperparameter Tuning

Lasso: a 25-point logarithmic sweep of the regularization strength (alpha), scored on validation MAE, selected alpha ≈ 0.000215 (6 of 21 coefficients non-zero, down from 13 at the untuned default).

XGBoost: a randomized search over 50 configurations across a 9-parameter grid, scored by 5-fold expanding-window time-series cross-validation on the training set only. Best configuration: max_depth = 4, min_child_weight = 5, learning_rate = 0.01, n_estimators = 200, subsample = 0.7, colsample_bytree = 0.5, gamma = 0.0, reg_alpha = 0.1, reg_lambda = 0.5 (mean CV RMSE = 0.011237).

LSTM: a controlled manual search over 5 configurations (units, dropout, learning rate, batch size), each with early stopping on validation loss. Best: 64 units, dropout 0.30, learning rate 0.0005, batch size 64 (validation loss 0.72277).

5. Performance Evaluation

5.1 Regression Metrics (Test Set)

Reported against a Zero-Return Baseline (always predicts zero return) as the honest floor for any "beats the market" claim.

Model	MAE	RMSE	R^2	Dir. Accuracy
Zero-Return Baseline	0.009195	0.013236	-0.005592	—
Lasso (Tuned)	0.009163	0.013197	0.000323	53.25%
LSTM (Tuned)	0.009123	0.013183	0.002461	56.44%
XGBoost (Tuned)	0.009191	0.013233	-0.005104	52.72%

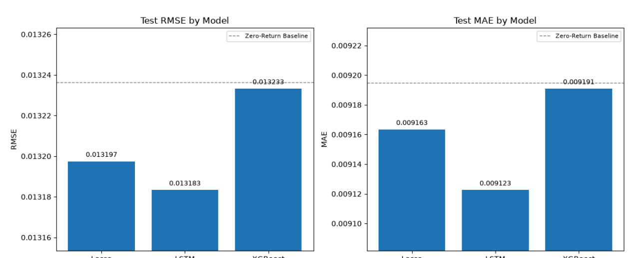


Figure 5.1 — Test RMSE and MAE by tuned model, with the Zero-Return Baseline shown as a reference line.

For the tuned LSTM, a two-proportion z-test with n=753 gives $z \approx 3.53$ and $p \approx 0.0004$, suggesting that its directional accuracy is significantly above 50%.

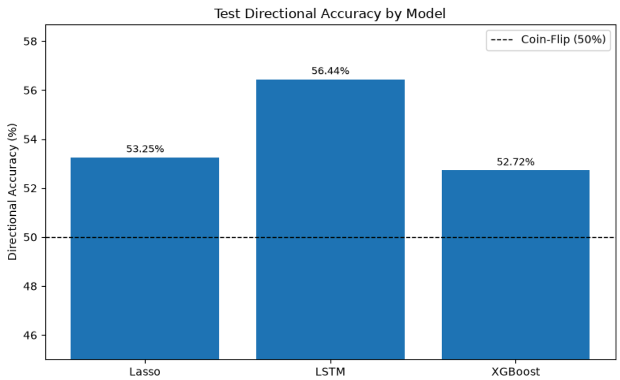


Figure 5.2 — Test directional accuracy by tuned model

5.2 Direction-as-Classification Framing

Each model's predicted return was also converted into a simple up-or-down classification. The test set was not perfectly balanced, with 423 positive-return days and 330 negative-return days. This means that a model predicting "up" every day would already achieve 56.2% accuracy.

Model	Accuracy	F1	ROC-AUC
Majority-Class Baseline	56.18%	0.7194	—
LSTM (Tuned)	56.44%	0.6911	0.5359
Lasso (Tuned)	53.25%	0.6207	0.5194
XGBoost (Tuned)	52.72%	0.6114	0.5112

The Majority-Class Baseline has the highest F1-score, even though it is not a meaningful predictive model. It simply predicts the more common class, "up," for every observation. Among the actual models, the tuned LSTM performs best, but it still relies heavily on predicting the majority class. Because of this imbalance, ROC-AUC provides a fairer comparison than accuracy or F1-score alone. However, performance remains weak

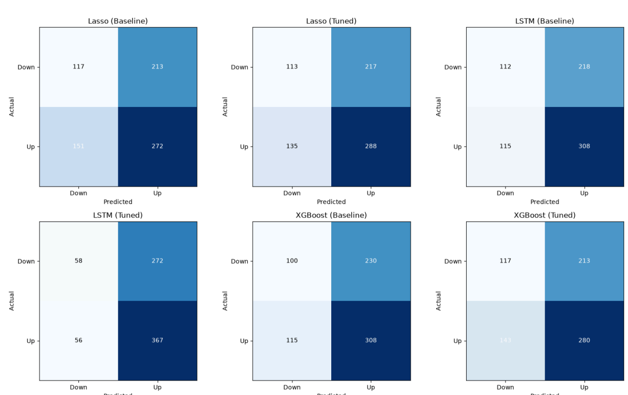


Figure 5.3 — Confusion matrices for the baseline and tuned versions of Lasso, LSTM, and XGBoost on the 753-day test set.

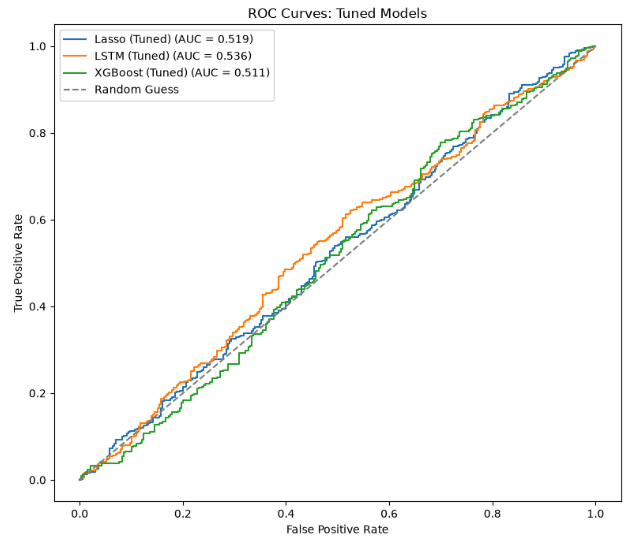


Figure 5.4 — ROC curves for the tuned version of each model.

6. Feature Importance

The three models point to a similar group of important variables, even though they measure importance in different ways. Lasso uses coefficient size, XGBoost uses gain-based importance, and LSTM uses permutation importance because it does not provide simple coefficients. Across the models, changes in Treasury yields, 10-year breakeven inflation, and the U.S. dollar index tend to rank near the top. In contrast, raw gold trading volume and the one-day returns of platinum and copper usually rank near the bottom. Overall, the importance values are fairly small. For the LSTM, even shuffling the highest-ranked feature increased validation RMSE by less than 1%, which suggests that no single feature provides a strong signal on its own.

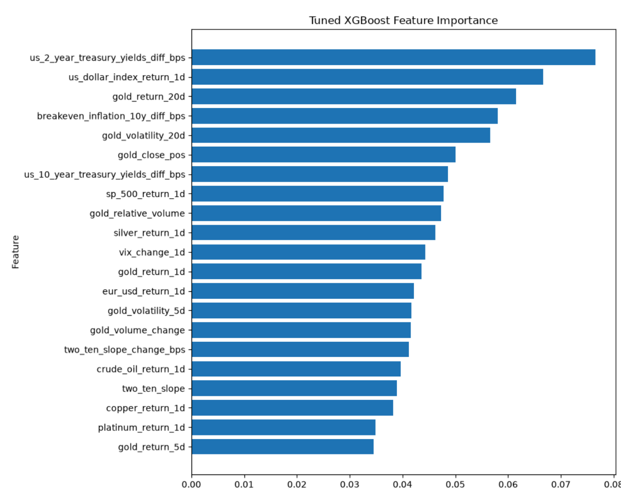


Figure 6.1 — Tuned XGBoost feature importance (gain-based, all 21 features), representative of the convergence across all three methods.

7. Research Question Answers

1. Which model predicts next-day gold returns most accurately?

The tuned LSTM performed best overall. It had the strongest regression results and also achieved the highest F1-score and ROC-AUC among the trained models. Its directional accuracy was significantly above 50% ($p \approx 0.0004$), although the advantage over the other models and the simple baselines was still fairly small.

2. Which features are most important?

The models generally placed the greatest importance on changes in Treasury yields, 10-year breakeven inflation, and the U.S. dollar index. The ablation results support this pattern: macroeconomic features improved performance, while the additional commodity variables did not. Overall, however, the effect sizes remained small, suggesting that no individual feature provides a strong standalone signal.

3. How does performance change with hyperparameter settings?

The effect of tuning depended on the model. It improved the LSTM, produced mixed results for Lasso, and did not improve XGBoost's performance on the test set.

4. Does macro data improve accuracy over gold price history alone?

Yes, although the improvement is small. The Gold-Only model produced a validation RMSE of 0.009862 and an R^2 of 0.002665. After adding macroeconomic variables, RMSE improved to 0.009791 and R^2 increased to 0.016937. Directional accuracy also rose slightly from 54.26% to 54.39%. This suggests that macroeconomic features add some useful information beyond gold's own price history, but the practical gain is limited.

5. Do other commodities (silver, oil) improve predictions?

The Gold + Commodities model performed slightly worse than the Gold-Only model, with RMSE increasing from 0.009862 to 0.009869 and R^2 falling to 0.001072. Adding the four commodity variables to the Gold + Macro model also produced no measurable improvement. Lasso reduced all four commodity coefficients to zero, indicating that they did not contribute additional predictive value once gold-history and macroeconomic features were already included.

8. Limitations

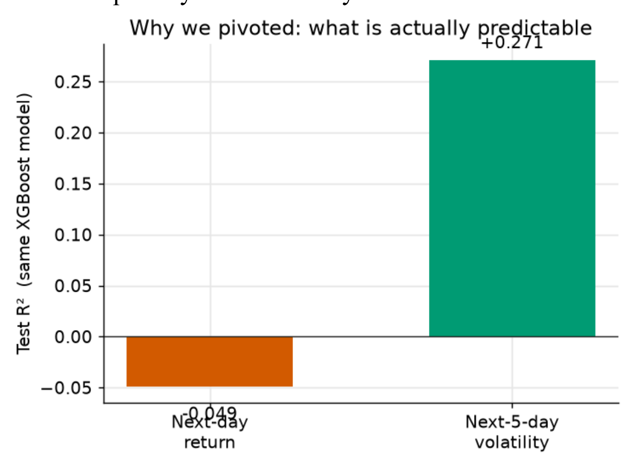
The dataset covers roughly 22 years, but the test set contains only one continuous 753-day period. As a result, the evaluation reflects a specific part of market history rather than a wide range of independently sampled market conditions.

The models also do not account for transaction costs, bid-ask spreads, or slippage. Therefore, even the LSTM's small directional advantage may not lead to a profitable trading strategy once realistic costs are included.

More broadly, the weak predictive performance is consistent with gold being a highly liquid and closely followed asset. At a daily frequency, much of the available information may already be reflected in market prices. This does not necessarily mean the modeling process failed; it may instead reflect the limited predictability of short-term gold returns.

9. Forecasting Gold Volatility

The results show us that predicting tomorrow's gold move is close to unpredictable as they are too noisy, but volatility is persistent. Calm days and turbulent days cluster together (the well-documented ARCH/GARCH effect). We therefore tried re-pointed the XGBoost model at a new target while keeping the same data and feature set, to test whether the same pipeline can produce genuine skill on a quantity that is actually forecastable.



9.1 Target and Features

Target: The log of gold's average daily realized volatility over the next 5 trading days, measured with the Parkinson high-low range estimator. The Parkinson estimator uses the daily High/Low already in our dataset and extracts far more information than close-to-close returns. The log transform makes the right-skewed target near-Gaussian and prevents negative predictions.

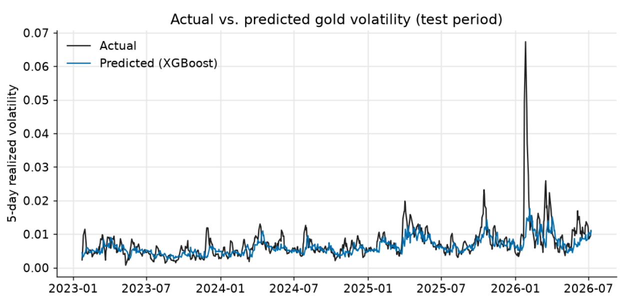
Features: We use three HAR (Heterogeneous Autoregressive) components additionally. The volatility averaged over the past day, week (5-day), and month (22-day).

9.2 Benchmarks and Results

Similar to the earlier model, this model is judged against two honest references: a persistence baseline (predict that next week's volatility equals last week's) and a linear HAR model (the canonical realized-volatility benchmark). We report RMSE, R^2, and QLIKE. These are the standard volatility-forecast loss, where lower is better.

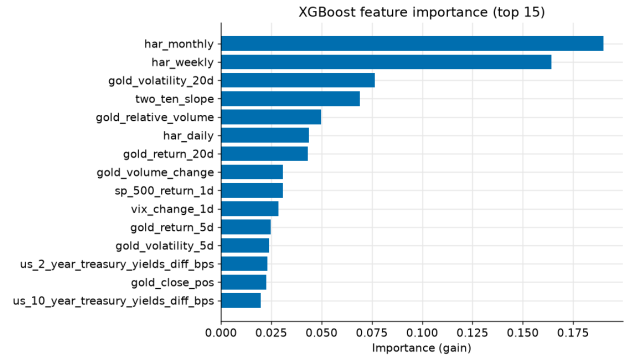
Model	MAE	RMSE	R^2	QLIKE
Persistence Baseline	0.003000	0.005453	-0.1634	0.7302
HAR (Linear)	0.002403	0.004515	0.2023	0.6671
XGBoost (Tuned)	0.002297	0.013233	0.2641	0.5331

Capturing approximately 26% of out-of-sample variance for the following week's gold volatility, the optimized XGBoost outperformed both the naive persistence baseline and the standard HAR benchmark across all evaluated metrics. Furthermore, this model achieved a 69.6% accuracy rate in forecasting whether volatility would increase or decrease.



9.3 Feature Importance

The gain-based importances are dominated by the HAR volatility components together with the trailing close-to-close volatility. The macroeconomic and cross-asset features play only a secondary, complementary role. No single macro feature carries a strong standalone signal.



10. GitHub Repository

https://github.com/khgaurav/macro-commodity-ml